

Kiln: A Self-Evolving Tool Registry for Autonomous AI Agents

CS5224 Cloud Computing — Preliminary Report

Shivansh (A0328697H) • Arya Bhosale (A0314484A) • Himanshu (A0314584B)
• Sahajpreet (A0313530W) • Chun Jie (A0252615J)

National University of Singapore • AY2025/26 Semester 2 • March 2026

1. Motivation and Business Case

1.1 Problem Statement

Modern AI agents built with frameworks such as LangChain, CrewAI, AutoGen, and Anthropic’s Model Context Protocol (MCP) are fundamentally limited by static, pre-defined toolsets. When an agent encounters a task requiring a capability that has not been built, fetching real-time stock prices, geocoding an address, analyzing the sentiment of a product review, the agent simply fails. Extending the agent’s capabilities today requires three sequential bottlenecks: (1) a developer must manually write the tool code, (2) a deployment cycle must register it in the agent framework, and (3) the agent server often requires a restart to make the new tool available. The consequence is that agent capability scales linearly with human engineering effort, creating a bottleneck that grows increasingly painful as enterprise adoption of agentic AI accelerates.

The problem is compounded by deep fragmentation across the AI tool ecosystem. Each agent framework maintains its own incompatible tool format, a weather tool written for LangChain cannot be used in CrewAI without being completely rewritten. There is no equivalent of npm, PyPI, or Docker Hub for AI agent tools: no central registry for discovery, no versioning for reproducibility, no trust signals for quality, and no mechanism for tools to be shared across frameworks or organizations. This fragmentation leads to massive duplication of effort across the industry.

1.2 Proposed Solution

Kiln is a cloud-native SaaS platform that provides AI agents with a self-healing, self-expanding tool ecosystem. When an agent encounters a task requiring a tool that does not exist, Kiln autonomously synthesizes it via LLM-powered code generation inside a gVisor-sandboxed Cloud Run instance, validates the generated code against a multi-stage quality gate (JSON schema validation, test fixture execution, dependency safety checks against a blocked-packages list), registers the tool in the platform’s registry, and hot-loads it at runtime via Python’s importlib, all with zero downtime and zero human intervention. Once created, the tool becomes permanently available to every agent that connects to the platform.

Crucially, Kiln exposes its entire tool library via the Model Context Protocol (MCP, specification version 2025-11-25), the emerging open standard for how AI assistants discover and invoke external tools. Any MCP-compatible client — Claude Desktop, Cursor, VS Code Copilot, Windsurf, can connect to Kiln’s dedicated MCP Server (a separate Cloud Run microservice) and immediately access every tool in the registry. When new tools are synthesized, the MCP Server emits a notifications/tools/list_changed notification over the JSON-RPC protocol, prompting all connected AI clients to automatically refresh their tool catalogs without any manual intervention. This positions Kiln not as a standalone chat application, but as foundational infrastructure, the tool access layer for the agentic internet, analogous to how Context7 serves documentation to LLMs via MCP, but for executable capabilities rather than reference material.

1.3 Target Users and User Journey

- **Agent developers:** Connect Kiln’s MCP endpoint to their AI assistant by adding a single URL to their MCP client configuration. All tools in the registry are immediately auto-discovered. When the agent needs a tool that doesn’t exist, Kiln synthesizes it live. The developer never writes a tool integration manually again.
- **Tool authors (“GitHub for AI tools”):** Publish tools to the Kiln registry with full semantic versioning, tool lineage and fork tracking, live health dashboards showing execution success rates and latency, and “used by” social proof showing which agents and organizations rely on the tool. Authors can fork and improve community tools, building reputation.
- **Enterprise teams:** Deploy Kiln with private namespaces, role-based access control (RBAC), governance-as-code policies that are automatically enforced on every tool publish (e.g., “tools must not make external HTTP calls”), comprehensive audit logging, and Firebase Auth with Google/GitHub SSO for single sign-on.

1.4 Comparison with Existing Services

Feature	Kiln (Ours)	LangChain Hub	OpenAI Plugins
Auto-synthesis of missing tools	LLM code gen + gVisor sandbox	No	No
Framework-agnostic format	spec.yaml + impl.py + requirements.txt	LangChain only	OpenAI only

MCP protocol (native)	Dedicated Cloud Run MCP server with list_changed	No	No
Runtime hot-loading	Python importlib (zero restart)	Requires restart	Requires restart
Tool dependency management	Auto-generated requirements.txt + blocked packages	Manual	Manual
Governance-as-code	Org-scoped policies enforced on every publish	No	No
Self-hosted option	Local registry (SQLite + disk)	Cloud only	Vendor-locked

2. Business Model

2.1 Revenue Strategy (Freemium + Marketplace)

Kiln operates on a freemium model designed to minimize adoption friction while capturing value at scale:

- **Free Tier:** Unlimited tool execution from the public registry, up to 10 synthesis operations per month, a single-user local registry, access to the community MCP endpoint, and the browser-based Tool Playground for testing tools without installation.
- **Pro Tier (\$29/month):** Unlimited synthesis, a private tool namespace, team collaboration with up to 5 seats, an analytics dashboard with usage metrics, Agent Replay for step-by-step execution debugging, and priority execution on dedicated Cloud Run instances.
- **Enterprise Tier (custom pricing):** Dedicated GCP project with isolated infrastructure, SSO/SAML integration, comprehensive audit logging, SLA guarantees, governance-as-code policy enforcement, and an on-premise deployment option.
- **Marketplace (15% commission):** Tool authors can set per-execution pricing on premium tools (e.g., tools wrapping expensive APIs or proprietary algorithms). Kiln handles all billing, usage metering, and revenue sharing.

2.2 Cloud vs On-Premise Cost Comparison

Component	Cloud (GCP)	On-Premise Equivalent
Compute (5 services)	Cloud Run: ~\$28/mo (scale-to-zero, pay-per-request)	3x bare-metal servers: ~\$500/mo + power + cooling
Frontend + Auth	Firebase Hosting + Auth: \$0 (free up to 10K MAU)	Self-hosted React + Keycloak: ~\$100/mo + ops
Database (2 instances)	Cloud SQL (shared-core PostgreSQL): ~\$8/mo	Self-managed PostgreSQL: ~\$150/mo HW + DBA time
Object Storage	GCS: ~\$1/mo (per-user tool artifacts)	MinIO on local disks: ~\$50/mo + no CDN
Observability	Cloud Logging + Trace + Monitoring: \$0 (free tier)	Self-hosted ELK + Prometheus: ~\$100/mo + ops
CI/CD + Registry	Cloud Build + Artifact Registry: ~\$1/mo	Jenkins + Harbor: ~\$50/mo + maintenance
Total	~\$38/month (at 100 users)	~\$950+/month + significant ops overhead

The cloud deployment delivers a 96% cost reduction compared to on-premise while completely eliminating infrastructure management burden. Cloud Run's pay-per-request billing model is particularly transformative at startup scale: five independently deployable services with full scale-to-zero capability cost approximately \$28/month combined, compared to \$500+ for always-on servers that must run 24/7 regardless of whether any user is active.

3. Architecture and Implementation

3.1 System Architecture Overview

Figure 1 presents the complete system architecture deployed on Google Cloud Platform. The platform uses Cloud Run for all compute workloads (five independent microservices), Firebase for user authentication and static frontend hosting, Cloud Pub/Sub for asynchronous inter-service messaging, and GCP's managed data services for persistence and observability.

[illegible]

- **MCP Server as a dedicated service:** The MCP Server is architecturally a thin, stateless adapter that translates JSON-RPC messages into REST calls against the Registry API. Separating it as its own Cloud Run service allows independent scaling (MCP traffic patterns from AI assistants differ substantially from human web traffic), and keeps the Registry API as a clean REST service that also serves the web UI and CLI clients.
- **Firebase Auth as shared identity:** Firebase Auth issues JWTs that all five backend services verify via cached public key validation (no network call to Firebase needed). This provides a unified identity across both the Chat UI and Registry UI. Firebase Auth is free up to 10,000 monthly active users and supports email/password and Google OAuth out of the box. On first login, each user is assigned a generated API key stored in Cloud SQL for CLI and MCP client authentication.
- **Pub/Sub for async synthesis:** The Chat Backend publishes synthesis requests to a Cloud Pub/Sub topic. The Synthesis Service subscribes, generates the tool artifacts, and fires a webhook callback to the Registry API. This fully decouples the synthesis lifecycle from the chat request lifecycle, ensuring that a slow or failed synthesis never blocks the user's SSE stream.
- **Two Cloud SQL instances:** The Chat DB (user accounts, encrypted key vault, session state) and Registry DB (tool metadata, version history, namespaces, download statistics) are separated into distinct Cloud SQL instances for failure isolation, a registry database outage should not affect user authentication, and vice versa.

3.4 Data Layer

Cloud SQL (PostgreSQL) with GIN indexes provides full-text search. Registry DB stores tool metadata, versions, and namespaces. Chat DB stores user accounts, the encrypted key vault, and session state. GCS stores immutable per-user tool artifacts (spec.yaml + impl.py + requirements.txt) with signed URLs (15-min TTL). Memorystore (Redis) is optional for caching and rate limiting. Secret Manager holds platform secrets injected via Cloud Run secret references.

3.5 GCP Service Map and Estimated Cost

GCP Service	Role in Kiln	Monthly Cost
Cloud Run (5 services)	All backend compute: Chat, Registry, MCP, Executor, Synthesis	~\$28
Firebase Hosting (2 sites)	Static hosting for Chat UI and Registry UI (React SPAs)	\$0
Firebase Auth	Shared identity provider, JWT issuer, Google/GitHub OAuth	\$0
Cloud SQL (2x PostgreSQL)	Chat DB (accounts, vault, sessions) + Registry DB (tools, versions)	~\$8
Cloud Storage (GCS)	Per-user tool artifacts (spec.yaml + impl.py + requirements.txt)	~\$1
Cloud Pub/Sub	Async synthesis messaging (Chat Backend → Synthesis Service)	\$0
Secret Manager	Platform secrets (ANTHROPIC_API_KEY, DB credentials)	\$0
Cloud Logging + Trace + Monitoring	Structured logs, distributed tracing, dashboards, alerts, SLOs	\$0
Cloud Build + Artifact Registry	Per-service CI/CD pipelines, Docker image storage with scanning	~\$1
Memorystore (Redis) — optional	Warm cache, session state, rate limiting	\$0–\$35
Total (without Redis)		~\$38/mo

4. Evaluation Plan

4.1 Functional Evaluation

- End-to-end tool synthesis: Submit 20 diverse natural language queries that require tools not present in the registry. Measure synthesis success rate (target: >80%), time from request to tool availability (target: <60 seconds), and correctness of execution results against expected outputs.
- MCP protocol compliance: Connect Claude Desktop, Cursor, and VS Code Copilot to Kiln's MCP Server endpoint. Verify that tools/list returns the complete tool catalog, tools/call correctly executes tools and returns structured results, and notifications/tools/list_changed triggers automatic catalog refresh in all three clients after a new tool is synthesized.
- Chat application demo (primary demo for video presentation): Submit natural language queries through the React Chat UI. Verify DAG visualization renders correctly, tool execution streams results via SSE in real time, and the final answer is coherent and accurate.

4.2 Non-Functional Evaluation

- Scalability: Load test with k6 simulating 50 to 500 concurrent users. Measure Cloud Run auto-scaling behavior, cold start latency for each service, request throughput at saturation, and Pub/Sub message processing latency under load.

- Security: Attempt container escape from gVisor-sandboxed Tool Executor, verify the blocked-packages list prevents dangerous imports (e.g., subprocess, pyautogui), test Firebase JWT validation edge cases (expired tokens, malformed tokens, cross-project tokens).
- Cost efficiency: Compare actual GCP billing over a 2-week operational period against the on-premise cost estimates with identical synthetic workloads.
- High availability: Verify zero-downtime rolling updates across all five Cloud Run services using revision-based traffic splitting. Simulate Cloud SQL failover and verify the application degrades gracefully rather than failing completely.

5. Implementation Plan and Timeline

Week	Phase	Key Deliverables
1–2	Infrastructure	Nx monorepo scaffolding (FastAPI + React + MCP server packages). GCP provisioning via Terraform: Cloud Run services, Cloud SQL instances, GCS buckets, Pub/Sub topics. Firebase project setup (Auth + Hosting). Cloud Build CI/CD pipelines for all 7 targets.
2–3	Registry + Executor	Registry API: CRUD, full-text search, push/pull, RBAC auth, version management. Tool Executor: gVisor sandbox, requirements.txt pip install, blocked packages validation. spec.yaml format finalized. Quality gate: schema validation + test fixture execution.
3–4	Chat + Synthesis	Chat Backend: LLM Planning Agent, Graph Executor (Kahn’s algorithm), SSE streaming, User Key Vault. Synthesis Service: Claude API integration, spec + impl + requirements generation, Pub/Sub subscription, webhook callback. React Chat UI with live DAG visualization. End-to-end demo working.
4–5	MCP + Registry UI	MCP Server: JSON-RPC ↔ REST adapter, tools/list + tools/call + list_changed. Registry UI: tool detail pages, search, Tool Playground, publish workflow, version history. Firebase Auth integration across both UIs. API key generation for CLI/MCP clients.
5–6	Polish + Evaluation	Observability: OpenTelemetry integration, Cloud Trace, dashboards, alert policies, SLOs. k6 load testing (50–500 users). gVisor security audit. User testing with 5–10 NUS CS students (SUS scores). Cost analysis. Documentation. 15-minute video presentation with live demo.

6. Conclusion

Kiln addresses the fundamental bottleneck in agentic AI: the dependence on static, manually-built tool ecosystems that cannot keep pace with the growing demand for agent capabilities. By combining autonomous LLM-powered tool synthesis with a production-grade Cloud Run architecture consisting of five independently scalable microservices, the platform transforms tool creation from a human engineering constraint into an elastic, self-healing cloud operation. The dedicated MCP Server makes Kiln infrastructure that plugs directly into the existing AI assistant ecosystem, users do not install individual tools, they connect to a single endpoint and every tool in the registry, including tools that have not been created yet, becomes immediately available. Cloud Run’s pay-per-request billing model keeps the startup cost at approximately \$38 per month for 100 users, representing a 96% reduction compared to an equivalent on-premise deployment. The five-week implementation timeline is achievable given the team’s existing familiarity with the core technologies, the Nx monorepo structure enabling parallel development across all services, and Cloud Run’s elimination of cluster management overhead.

References

[1] Anthropic, “Model Context Protocol Specification v2025-11-25,” modelcontextprotocol.io, Nov 2025.

[2] Google Cloud, “Cloud Run: Fully Managed Serverless Platform,” cloud.google.com/run/docs, 2025.

[3] Google, “gVisor: Application Kernel for Containers,” gvisor.dev, 2024.

[4] Google, “Firebase Authentication Documentation,” firebase.google.com/docs/auth, 2025.

[5] OpenTelemetry Authors, “Python SDK for Distributed Tracing,” opentelemetry.io/docs/languages/python, 2025.

[6] Google Cloud, “Cloud Pub/Sub: Messaging and Event Ingestion,” cloud.google.com/pubsub/docs, 2025.

[7] Nrwl, “Nx: Smart, Fast, Extensible Build System,” nx.dev, 2025.

[8] Google Cloud, “Cloud Build: Serverless CI/CD Platform,” cloud.google.com/build/docs, 2025.